

# AI Meeting & Follow-Up Agent

Implementation Plan — Including Real-Time WebSocket Pipeline  
InnovaHack Chapter-1 | Domain 4: Agentic AI | Problem Statement 2

## 1. Project Overview

This plan outlines how to build a working, demoable AI Meeting & Follow-Up Agent within a 24-hour hackathon window. The system ingests a meeting transcript, extracts decisions and action items using an LLM, assigns owners and deadlines, tracks status on a live dashboard, and autonomously simulates reminders until items are marked complete. A WebSocket layer is used to push real-time updates from the backend processing pipeline to the frontend dashboard, so the extraction and reminder events appear live on screen during the demo.

## 2. Goals for the Hackathon Build

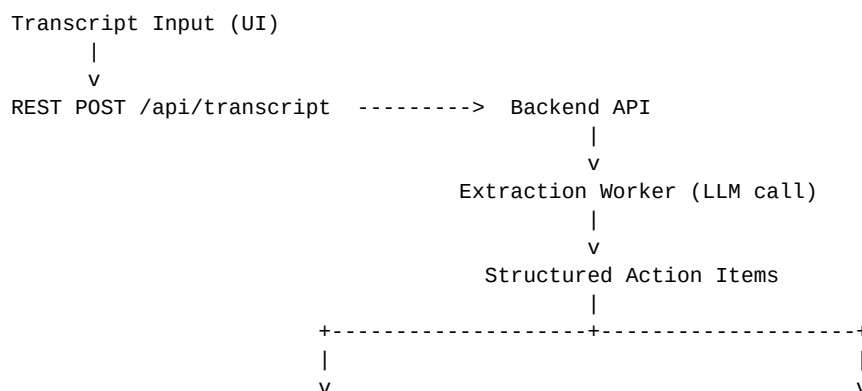
- Deliver a complete end-to-end flow: transcript in → structured action items out → live dashboard → simulated reminders.
- Use WebSockets to stream extraction progress and reminder events to the UI in real time (not polling).
- Keep scope realistic: use a pasted/uploaded text transcript rather than live audio transcription.
- Make the demo visually convincing: judges should see items appear, get assigned, and receive a simulated follow-up live.

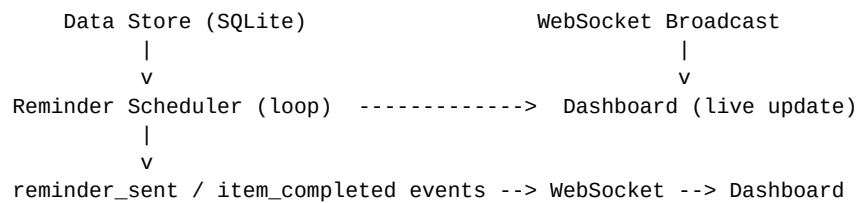
## 3. System Architecture

High-level components:

- **Frontend (Dashboard):** React app showing transcript input, live action-item list, owner/deadline/status columns, and a reminder feed.
- **Backend API server:** FastAPI (Python) or Node/Express, exposes REST endpoints for transcript submission and a WebSocket endpoint for live updates.
- **Extraction Worker:** Calls the LLM (Claude/GPT) with a structured prompt to pull decisions, action items, owners, and deadlines from the transcript.
- **Reminder Scheduler:** A lightweight in-process scheduler (or simulated timer) that checks pending items and triggers reminder events at set intervals.
- **Data Store:** SQLite or in-memory store (dictionary/JSON file) — sufficient for a hackathon; no need for a production database.
- **WebSocket Layer:** Pushes events (item\_extracted, item\_updated, reminder\_sent, item\_completed) from backend to all connected dashboard clients.

Architecture flow:





## 4. Real-Time Pipeline Using WebSockets

WebSockets are used so the dashboard updates live as the backend works, instead of the frontend repeatedly polling the server. This is what makes the demo feel like an active "agent" rather than a static form submission.

### 4.1 Why WebSockets Here

- Extraction can take a few seconds (LLM call) — a WebSocket lets the UI show a live "processing" state and stream items in as they're identified, rather than a blocking spinner.
- Reminders are asynchronous and time-based — the server needs to push "reminder sent" events to the UI at the moment they happen, not when the browser happens to ask.
- Multiple dashboard views (e.g., a judge's screen and your own) can stay in sync automatically.

### 4.2 Connection Flow

- On dashboard load, the frontend opens a WebSocket connection to `ws://<server>/ws/meeting/{meeting_id}`.
- The server registers the connection in an active-connections list for that meeting.
- When the transcript is submitted (via REST POST), the backend starts the extraction worker asynchronously and begins emitting events over the WebSocket.
- The frontend listens for events and updates the UI incrementally as they arrive.

### 4.3 Event Types (Message Schema)

All messages are sent as JSON with a **type** field so the frontend can route them:

```

{ "type": "processing_started" }

{ "type": "item_extracted",
  "data": { "id": "1", "task": "Send updated proposal",
            "owner": "Riya", "deadline": "2026-07-28",
            "status": "pending" } }

{ "type": "processing_complete", "total_items": 5 }

{ "type": "reminder_sent",
  "data": { "id": "1", "channel": "email (simulated)",
            "timestamp": "2026-07-26T10:15:00Z" } }

{ "type": "item_completed", "data": { "id": "1" } }

```

### 4.4 Backend Implementation Sketch (FastAPI)

```

from fastapi import FastAPI, WebSocket
import asyncio, json

app = FastAPI()
connections = {} # meeting_id -> list of WebSocket

@app.websocket("/ws/meeting/{meeting_id}")
async def ws_endpoint(websocket: WebSocket, meeting_id: str):
    await websocket.accept()
    connections.setdefault(meeting_id, []).append(websocket)
    try:
        while True:
            await websocket.receive_text() # keep-alive / client pings
    except Exception:

```

```

        connections[meeting_id].remove(websocket)

    async def broadcast(meeting_id: str, message: dict):
        for ws in connections.get(meeting_id, []):
            await ws.send_text(json.dumps(message))

    @app.post("/api/transcript/{meeting_id}")
    async def submit_transcript(meeting_id: str, payload: dict):
        asyncio.create_task(process_transcript(meeting_id, payload["text"]))
        return {"status": "processing_started"}

    async def process_transcript(meeting_id: str, text: str):
        await broadcast(meeting_id, {"type": "processing_started"})
        items = await extract_action_items(text)  # LLM call
        for item in items:
            save_item(meeting_id, item)
            await broadcast(meeting_id, {"type": "item_extracted", "data": item})
            await asyncio.sleep(0.3)  # slight stagger for visual effect in demo
        await broadcast(meeting_id, {"type": "processing_complete",
                                      "total_items": len(items)})
        asyncio.create_task(reminder_loop(meeting_id))

```

## 4.5 Reminder Loop (Simulated)

The reminder loop runs as a background task per meeting. For the demo, intervals should be compressed (e.g., every 30-60 seconds) rather than real-world day-long gaps, so judges can see the loop in action without waiting.

```

    async def reminder_loop(meeting_id: str):
        while True:
            await asyncio.sleep(30)  # compressed interval for demo
            pending = get_pending_items(meeting_id)
            if not pending:
                break
            for item in pending:
                mark_reminder_sent(item["id"])
                await broadcast(meeting_id, {
                    "type": "reminder_sent",
                    "data": {"id": item["id"], "channel": "email (simulated)",
                           "timestamp": now_iso()}
                })

```

## 4.6 Frontend Implementation Sketch (React)

```

useEffect(() => {
    const ws = new WebSocket(`ws://localhost:8000/ws/meeting/${meetingId}`);

    ws.onmessage = (event) => {
        const msg = JSON.parse(event.data);
        switch (msg.type) {
            case "processing_started":
                setStatus("Extracting action items...");
                break;
            case "item_extracted":
                setItems((prev) => [...prev, msg.data]);
                break;
            case "processing_complete":
                setStatus(`Done - ${msg.total_items} items found`);
                break;
            case "reminder_sent":
                addToReminderFeed(msg.data);
                break;
            case "item_completed":
                markItemComplete(msg.data.id);
                break;
        }
    };

    return () => ws.close();
}, [meetingId]);

```

## 5. Recommended Tech Stack

| Layer      | Recommended                              | Notes                                          |
|------------|------------------------------------------|------------------------------------------------|
| Frontend   | React + Tailwind                         | Fast to build, clean dashboard styling         |
| Backend    | FastAPI (Python)                         | Native async support, easy WebSocket handling  |
| Real-time  | Native WebSockets (FastAPI) or Socket.IO | Socket.IO adds auto-reconnect if time allows   |
| LLM        | Claude or GPT API                        | Structured JSON output for extraction          |
| Data store | SQLite / in-memory dict                  | No need for a full production DB               |
| Deployment | Render / Railway / local + ngrok         | Any quick deploy option for the live demo link |

## 6. Data Model

```
ActionResult {
  id: string
  meeting_id: string
  task: string
  owner: string
  deadline: date | null
  priority: "low" | "medium" | "high"
  status: "pending" | "reminded" | "completed"
  reminders_sent: integer
  created_at: datetime
}
```

## 7. Extraction Prompt Design

The extraction step should instruct the LLM to return strictly structured JSON so it can be parsed directly into the data model above. Example system instruction:

```
You will be given a meeting transcript. Extract all action items
and decisions. For each action item, identify:
- task (short description)
- owner (person responsible, or "unassigned" if unclear)
- deadline (explicit date/timeframe mentioned, or null)
- priority (infer from urgency language, default "medium")
```

```
Return ONLY valid JSON: a list of objects with keys
task, owner, deadline, priority. No extra commentary.
```

## 8. 24-Hour Build Timeline

| Time Block | Focus                                                                                    |
|------------|------------------------------------------------------------------------------------------|
| Hour 0-2   | Finalize scope, set up repo, design data model, sample transcripts prepared              |
| Hour 2-5   | Backend skeleton: REST endpoint, DB/schema, LLM extraction function tested standalone    |
| Hour 5-8   | WebSocket server implemented; broadcast events wired to extraction flow                  |
| Hour 8-12  | Frontend dashboard built: transcript input, live item list, status columns               |
| Hour 12-15 | Frontend WebSocket client integrated; live updates rendering correctly                   |
| Hour 15-18 | Reminder loop implemented (simulated, compressed interval) and connected end-to-end      |
| Hour 18-21 | Polish UI, add "mark complete" action, error handling, edge cases (no items found, etc.) |
| Hour 21-23 | Full run-through with 2-3 sample transcripts; fix bugs; deploy to public URL             |
| Hour 23-24 | Prepare slides (6-7), record backup demo video, final rehearsal                          |

## 9. Risks & Mitigations

| Risk                                         | Mitigation                                                           |
|----------------------------------------------|----------------------------------------------------------------------|
| LLM returns malformed JSON                   | Use strict prompt + a JSON-repair/retry step; validate before saving |
| WebSocket disconnects during demo            | Add simple auto-reconnect logic on the frontend                      |
| Live audio transcription unreliable          | Use pre-written sample transcripts for the demo instead              |
| Reminder loop timing looks unconvincing live | Compress intervals (seconds, not hours) during demo                  |
| Owner assignment inaccurate                  | Allow manual reassignment in the UI as a fallback                    |

## 10. Suggested Live Demo Script

- Paste in a sample meeting transcript and hit submit.
- Show the WebSocket-driven live extraction: items appearing one by one on the dashboard as "processing\_started" -> "item\_extracted" events stream in.
- Point out owner and deadline auto-assignment per item.
- Fast-forward the reminder loop (compressed interval) so a "reminder\_sent" event visibly appears in the feed.
- Mark one item complete to show status updates live across the dashboard.
- Close with the impact statement: no manual chasing, decisions never fall through the cracks.

This plan is intentionally scoped to be fully buildable within a 24-hour hackathon window while still demonstrating a genuine real-time, event-driven architecture via WebSockets.